actions.py

```python
from collections import deque
from point import Point
import numpy as np

def rect(buffer, start: Point, stop: Point, color, fill = None, maskBuffer = None):

    x1, x2 = sorted([start.x, stop.x])
    y1, y2 = sorted([start.y, stop.y])

    buffer[y1 : y2 + 1, x1 : x2 + 1] = color

    if not isinstance(maskBuffer, type(None)):
        maskBuffer[y1 : y2 + 1, x1 : x2 + 1] = True


def line(buffer, start: Point, stop: Point, color, maskBuffer = None):

    steps = max(np.abs(np.array((start.x - stop.x, start.y - stop.y)))) + 1

    # linear interpolation

    points = np.round(
        np.linspace(
            (start.x, start.y),
            (stop.x, stop.y),
            steps
    )).astype(int)

    buffer[points[:, 1], points[:, 0]] = color

    if not isinstance(maskBuffer, type(None)):
        maskBuffer[points[:, 1], points[:, 0]] = True

def ellipse(buffer, start: Point, stop: Point, color, fill: bool, maskBuffer =
None):
    x1, x2 = sorted([start.x, stop.x])
    y1, y2 = sorted([start.y, stop.y])

    rx = (x2 - x1) // 2
    ry = (y2 - y1) // 2
    cx = x1 + rx
    cy = y1 + ry

    rx2 = rx * rx
    ry2 = ry * ry

    x = 0
```

```
    y = ry

    dx = 2 * ry2 * x
    dy = 2 * rx2 * y

    def plot(px, py):
        if fill:
            # draw horizontal spans
            for fx in range(cx - px, cx + px + 1):
                buffer[cy + py, fx] = color
                buffer[cy - py, fx] = color
                if not isinstance(maskBuffer, type(None)):
                    maskBuffer[cy + py, fx] = True
                    maskBuffer[cy - py, fx] = True
        else:
            points = [
                (cx + px, cy + py),
                (cx - px, cy + py),
                (cx + px, cy - py),
                (cx - px, cy - py),
            ]
            for px_, py_ in points:
                buffer[py_, px_] = color
                if not isinstance(maskBuffer, type(None)):
                    maskBuffer[py_, px_] = True

    # --- Region 1 ---
    p1 = ry2 - rx2 * ry + 0.25 * rx2

    while dx < dy:
        plot(x, y)

        if p1 < 0:
            x += 1
            dx += 2 * ry2
            p1 += dx + ry2
        else:
            x += 1
            y -= 1
            dx += 2 * ry2
            dy -= 2 * rx2
            p1 += dx - dy + ry2

    # --- Region 2 ---
    p2 = (ry2 * (x + 0.5)**2 +
        rx2 * (y - 1)**2 -
        rx2 * ry2)

    while y >= 0:
```

```
        plot(x, y)

        if p2 > 0:
            y -= 1
            dy -= 2 * rx2
            p2 += rx2 - dy
        else:
            y -= 1
            x += 1
            dx += 2 * ry2
            dy -= 2 * rx2
            p2 += dx - dy + rx2

def floodFill(buffer, point: Point, color):

    targetColor = tuple(buffer[point.y, point.x])

    # Either current color, or if over network: provided color
    newColor = color

    if targetColor == newColor:
        return

    width, height, _ = buffer.shape

    def onCanvas(x, y):
        return 0 <= x < width and 0 <= y < height

    queue = deque()
    queue.append(point)

    while queue:
        x, y = queue.popleft()

        if not onCanvas(x, y):
            continue

        if not tuple(buffer[y, x]) == targetColor:
            continue

        buffer[y, x] = newColor

        queue.append((x - 1, y))
        queue.append((x + 1, y))
        queue.append((x, y + 1))
        queue.append((x, y - 1))
```